# Introduction to Unit Testing

Daniel Rodriguez

University of Alcala

# Outline


1. Introduction
2. Test Driven Development
3. Assertions
4. JUnit
5. First Example
6. Advanced Topics
7. Testing with Mock Objects
8. BDD and Cucumber
9. Web UI Testing
10. Code Coverage
11. Mutation Testing
12. Testing Strategy and Quality

1 Introduction
2 Test Driven Development
3 Assertions
4 JUnit
5 First Example
6 Advanced Topics
7 Testing with Mock Objects
8 BDD and Cucumber
9 Web UI Testing
10 Code Coverage
11 Mutation Testing
12 Testing Strategy and Quality

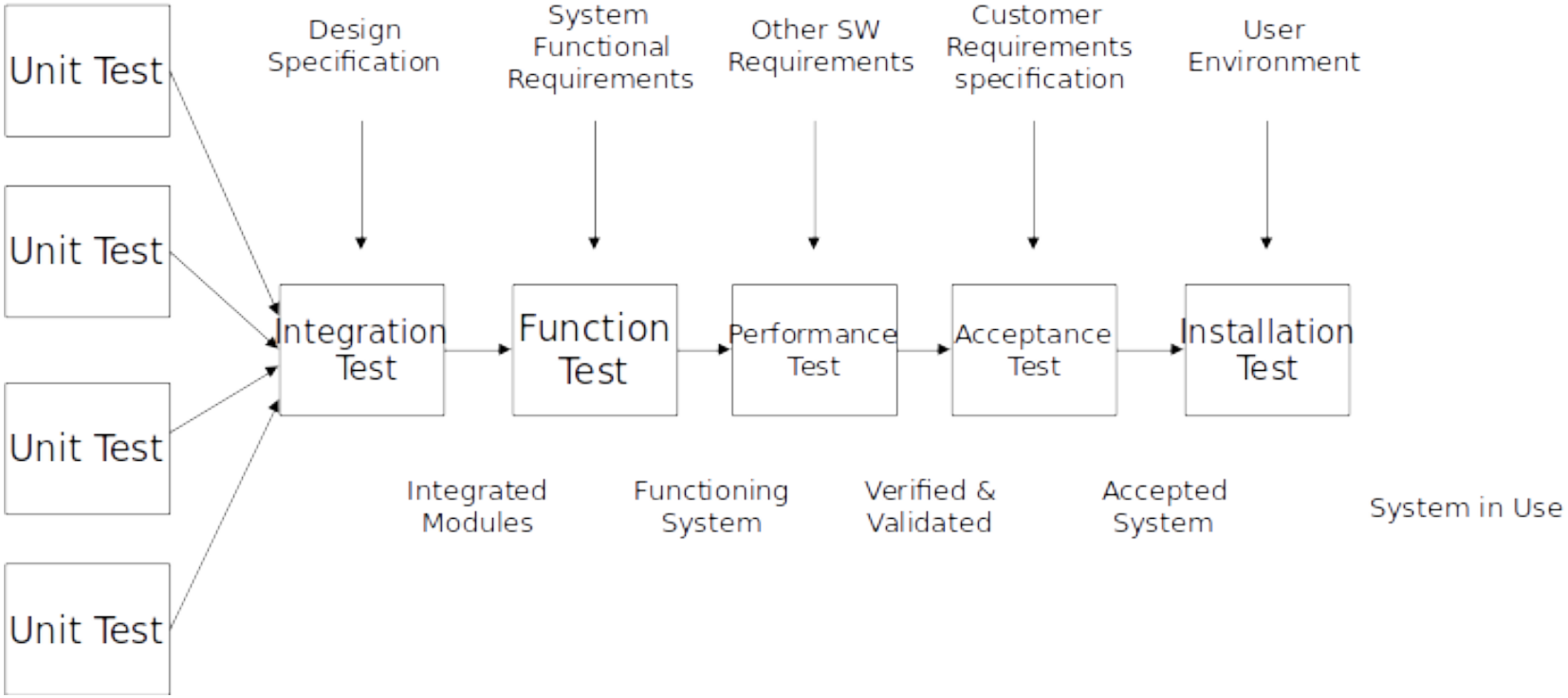
# Testing

- Error: A human mistake.
- Fault: The result of a mistake, evidenced in some development or maintenance product.
- Failure: A departure from the system's required behaviour.
- Fault identification (testing): Testing is the process of showing the presence of a software error. What fault caused the failure?
- Fault correction or removal (debugging): Debugging is the process of discovering the location of an error and its subsequent modification, i.e., changing the system to remove the fault.

# Verification vs. Validation - IEEE Std610.12-1990

- Requirements Validation. Process of checking that a document or piece of software accurately reflects the customer requirements - i.e. ARE WE BUILDING THE RIGHT SYSTEM?
- Software Verification. Process of ensuring that the product of one phase satisfies the product of a previous phase. sChecking that the software produced is a faithful implementation of the requirements specifiedAssessment of the quality of the implementation
  ARE WE BUILDING IT RIGHT?

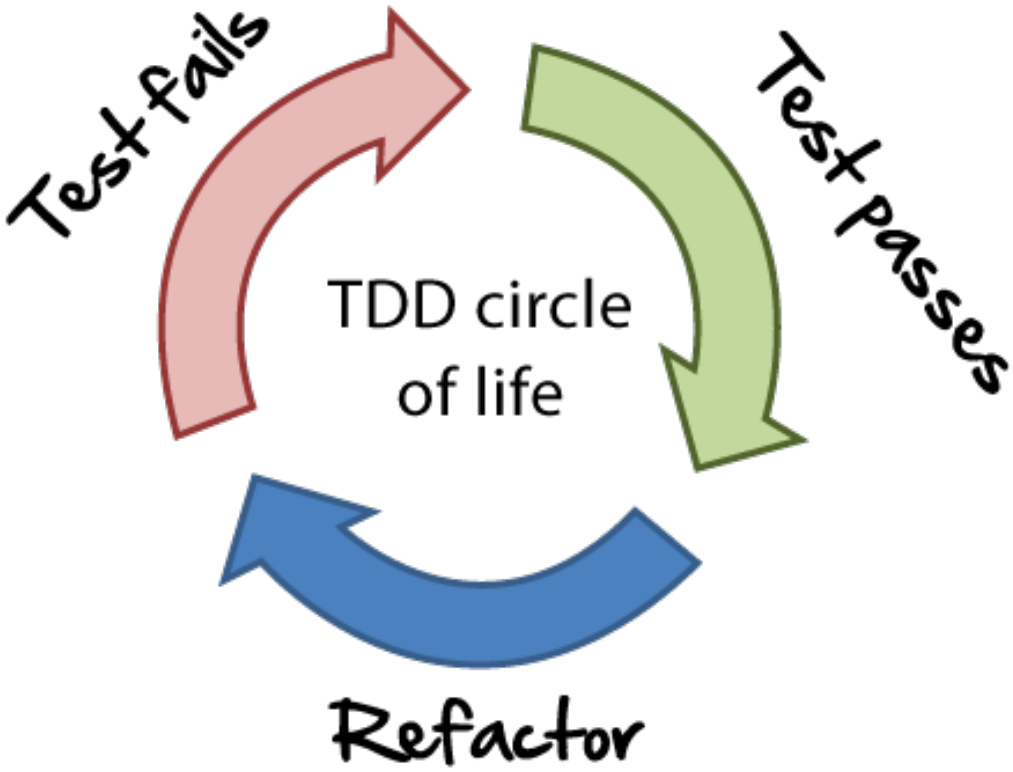
# Testing Process




(Source: Plfeeger)

1 Introduction
2 Test Driven Development
3 Assertions
4 JUnit
5 First Example
6 Advanced Topics
7 Testing with Mock Objects
8 BDD and Cucumber
9 Web UI Testing
10 Code Coverage
11 Mutation Testing
12 Testing Strategy and Quality

# Fail, Pass, Refactor: TDD Circle of life




**Source**: https://leantesting-wp.s3.amazonaws.com/resources/wp-content/uploads/2015/02/tdd-circle-of-life.png

# Fail, Pass, Refactor: TDD Circle of life

## TDD Life-cycle

1. Identify functionality to include
2. Write failing test
3. Make it compile as quickly as possible
4. Make it pass a quickly as possible
5. Refactor: remove duplication while maintaining 100% pass rate
6. Repeat as required

1 Introduction
2 Test Driven Development
3 Assertions
4 JUnit
5 First Example
6 Advanced Topics
7 Testing with Mock Objects
8 BDD and Cucumber
9 Web UI Testing
10 Code Coverage
11 Mutation Testing
12 Testing Strategy and Quality

# What are Assertions?

An assertion specifies a constraint over some state of computation. If it evaluates to false, it implies there exists an incorrect state in the program. Assertions:

- can test whether the execution state is consistent at the point of examination.
- are predicates involving the state variables of a program in execution.
- are embedded in the program text.

Assertions are part of most programming languages.

# Assertions: History

Hoare and Floyd started in the 60s (Hoare logic) the formal verification of programs. They introduced the concept of *preconditions*, *postconditions* and *invariants*.

A triple describes how the execution of a piece of code changes the state of the computation:

$[\{P\}C\{Q\}]$

where $P$ and $Q$ are assertions and $C$ is code.

- $P$ precondition
- $Q$ postcondition

In other words, a program behaviour can be verified using preconditions, postconditions and invariants.

# Assertions in Java

Assertions in Java are written as:

```
assert expression;
```

`assert` is a Java keyword and `expression` is any Boolean expression. When evaluated to `false`, an exception, `AssertionError`, will be raised.
They can also provide information for developers:

```
assert expression1 : expression2;
```

# Assert Example

```
public double avgMarks(double[] marks){
// Pre-condition
assert marks.length > 0 : "Array is empty!";
  double avg = 0;
  for (int i = 0 ; i < marks.length; i++){
     //Invariant
        assert marks[i] >= 0 && marks[i] <= 10 : "Mark is not
            valid!";
        avg += marks[i];
  }
  return avg/marks.length;
 }
```

# Assert example

```
public class AssertionExample {
  public double avgMarks(double[] marks){
    assert marks.length > 0 : "Array is empty!";
    double avg = 0;
    for (int i = 0; i < marks.length; i++) {
        assert marks[i] >= 0 && marks[i] <= 10 : "Mark is not
            valid!";
        avg += marks[i];
    }
    return avg/marks.length;
   }

  public static void main(String args[]){
    AssertionExample e = new AssertionExample();
    double[] marks = {10, 5, 11, 3, 2, 7.5};
    System.out.println(e.avgMarks(marks));
    marks = new double[0];
    System.out.println(e.avgMarks(marks));
   }
}
```

# Assert example

When assertions are evaluated when enabled, i.e., with the VM options `-enableassertions` or `-ea`:

```
$java -ea AssertionExample
```

```
drg@t500: ~/NetBeansProjects/AssertionExample/build/classes
drg@t500:~/NetBeansProjects/AssertionExample/build/classes$
drg@t500:~/NetBeansProjects/AssertionExample/build/classes$ java -ea assertionexample.AssertionExample
Exception in thread "main" java.lang.AssertionError: Nota no válida
        at assertionexample.AssertionExample.avgMarks(AssertionExample.java:15)
        at assertionexample.AssertionExample.main(AssertionExample.java:28)
drg@t500:~/NetBeansProjects/AssertionExample/build/classes$
```

Otherwise, it will run normally as assertions are designed to be run during the testing phase or debugging.

# Assertions vs Exceptions

Assertions are designed to check the program's correctness, i.e, as *preconditions*, *postconditions* or *invariants*.

Assertions can be used to verify the state of a program but should NOT be used to:

- Validate arguments of public methods
- Substitute the **exception handling** mechanims of a programming language
- Modify values of attributes or objects
- Be used in production code (assertions abruptly stop the execution of a program)

Exception handling is the process of responding to the occurrence of exceptions (usually in a graceful manner so that the execution is not interrupted). In Java, this is carried out with `try() ... catch() ... finally` statements.

Although most programming languages provide assertions, unit testing is usually carried out with frameworks such as JUnit (Java) or pytest (Python).

1 Introduction
2 Test Driven Development
3 Assertions
4 JUnit
5 First Example
6 Advanced Topics
7 Testing with Mock Objects
8 BDD and Cucumber
9 Web UI Testing
10 Code Coverage
11 Mutation Testing
12 Testing Strategy and Quality

# What is JUnit

https://www.junit.org/

- JUnit is the most popular open source unit testing framework in Java.
- JUnit is a regression testing framework.
- Created in 1997 by Erich Gamma and Kent Beck.
- JUnit 6 is based on the JUnit Platform and JUnit Jupiter.
- JUnit Vintage is optional and deprecated, only for temporary JUnit 3/4 migration.
- JUnit 6 requires Java 17 or later at runtime.
- Tests can be automated and grouped via test suites.
- *De facto* standard, integrated with IDEs, Maven, Gradle, and CI servers.

"Never in the field of software development have so many owed so much to so few lines of code" (Martin Fowler)

# Current JUnit Setup

For the examples in this course, use a current JUnit Jupiter setup:

- Java 17 or later to run the tests.
- Maven Surefire 3.5+.
- `org.junit:junit-bom:6.0.3` to keep JUnit modules aligned.
- `org.junit.jupiter:junit-jupiter` for new tests.
- Use `junit-vintage-engine` only as a temporary migration aid for JUnit 3/4 tests.

```
<properties>
  <maven.compiler.release>17</maven.compiler.release>
</properties>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.junit</groupId>
      <artifactId>junit-bom</artifactId>
      <version>6.0.3</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
```

# JUnit 4 (Legacy)

JUnit 4 introduced annotations and static imports and is still relevant when migrating older test suites.

- Static imports let us use assertion methods without qualifying the class name.
- JUnit 4 uses annotations instead of extending `TestCase`.
  - Annotations are metadata consumed by the test runner.
  - Naming conventions are no longer required, although they are still common.
- Common legacy annotations include:
  - Timeouts: `@Test(timeout=1000)`...
  - Testing exceptions: `@Test(expected=DBConnectException.class)`
  - Ignore a test: `@Ignore("Ignore the following test for now")`
- Legacy only: in JUnit Jupiter prefer `assertTimeout()`, `assertThrows()`, and `@Disabled`.

# JUnit 4 – Annotations

- `@Before` A method marked with `@Before` will be run before every test method. It is used to prepare the tests environment.
- `@After`
- `@Test` Identifies a test method.
- `@Ignore` The method under test will be ignored (to disable some tests which we do not want to repeat)
- `@AfterClass` Run only once after all test methods
- `@BeforeClass`

# JUnit Jupiter – Annotations

- `@BeforeEach` A method marked with `@BeforeEach` will be run before every test method. It is used to prepare the tests environment.
- `@AfterEach` It will be run after every test method.
- `@Test` Identifies a test method.
- `@Disabled` Disables a test or a test class.
- `@AfterAll` Run only once after all test methods
- `@BeforeAll` Run only once before all test methods

# JUnit Jupiter – Assert methods

Assert methods are used to comparison between an expected value and an actual value to determine the success or failure of a test.

- `assertCONDITION(...)`
- `assertCONDITION(String message,...)` - The message is displayed when the assert method fails

For example:

- `assertEquals(<> expected, <> actual)`
- `assertSame(...)`
- `assertFalse(boolean condition)`
- `assertTrue(boolean condition)`
- `assertThrows(Exception.class, executable)`
- `assertAll(...)`
- Etc.

Further information in the API:
`org.junit.jupiter.api.Assertions`

# JUnit Jupiter – New functionality I

- Custom display names - The `@DisplayName` annotation allows us to provide custom display names for test classes and methods, which can include spaces, special characters, and emojis.
- Nested test classes - Non-static nested classes (i.e., inner classes) can serve as `@Nested` test classes for logical grouping of test cases.
- Tagging and filtering - It is possible to tag test classes and test methods with your identifiers using the `@Tag` annotation.
- Meta-annotations and composed annotations - This allows us to create a custom composed annotation that will inherit the semantics of its meta-annotations.

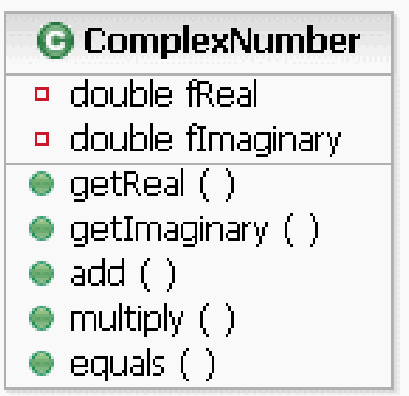
# JUnit Jupiter – New functionality II

- Dynamic tests - JUnit Jupiter introduces dynamic tests which are generated at runtime by a factory method annotated with the `@TestFactory` annotation.
- Repeated tests - Tests can be repeated a specified number of times by annotating the test method with the `@RepeatedTest` annotation.
- Parameterized tests - A test can be run multiple times with different arguments by annotating the test method with the `@ParameterizedTest` annotation.

1. Introduction
2. Test Driven Development
3. Assertions
4. JUnit
5. **First Example**
6. Advanced Topics
7. Testing with Mock Objects
8. BDD and Cucumber
9. Web UI Testing
10. Code Coverage
11. Mutation Testing
12. Testing Strategy and Quality

# ComplexNumber Class – Class Under Test (CUT)

We will use the `ComplexNumber` class as example, which will later be used to calculate quadratic equations.
The structure of the `ComplexNumber` class is as follows:




The operations are as follows:

- Addition: $(a, b) + (c, d) = (a + c) + (b + d)i$
- Multiplication: $(a, b) \cdot (c, d) = (ac - bd) + (ad + cb)i$
- Equality: $(a, b) = (c, d) \Leftrightarrow a = c \wedge b = d$

# ComplexNumber class – CUT I

```
public class ComplexNumber {
  private double realPart;
  private double imagPart;

  public ComplexNumber(double re, double im) {
    realPart = re;
    imagPart = im;
  }

  public double getRealPart() {
    return realPart;
  }

  public double getImaginaryPart() {
    return imagPart;
  }

public ComplexNumber add(ComplexNumber c) {
  return new ComplexNumber(getRealPart() + c.getRealPart(),
                           getImaginaryPart() + c.getImaginaryPart
                               ());
}
```

# ComplexNumber class – CUT II

```
public ComplexNumber multiply(ComplexNumber c) {
  double re = getRealPart() * c.getRealPart()
            - getImaginaryPart() * c.getImaginaryPart();
  double im = getImaginaryPart() * c.getRealPart()
            + getRealPart() * c.getImaginaryPart();
  return new ComplexNumber(re, im);
}

@Override
public boolean equals(Object anObject) {
  if (anObject instanceof ComplexNumber) {
    ComplexNumber c = (ComplexNumber) anObject;
    return ((c.getRealPart() == getRealPart()) &&
            (c.getImaginaryPart() == getImaginaryPart()));
  } else
    return false;
  }
}
}
```

# Fixtures

Now you can view your test class in the test source folder. Extract from test class:

```
public class ComplexNumberTest {
  @BeforeEach
  public void setUp()

  @AfterEach
  public void tearDown()

  ...
}
```

We can write some tests that start with the annotation `@BeforeEach` for the `setUp()` method as follows.

```
@BeforeEach
public void setUp() throws Exception {
  cOneZero = new ComplexNumber(1, 0);
  cZeroOne = new ComplexNumber(0, 1);
  cOneOne = new ComplexNumber(1, 1);
}
```

This method is executed before each test case is run. It is typically used to create class instances.

We can destroy all previously created classes with the annotation `@AfterEach` before a method, usually named `tearDown()`.

```
@AfterEach
public void tearDown() throws Exception {
  cOneZero = null;
  cZeroOne = null;
  cOneOne = null;
}
```

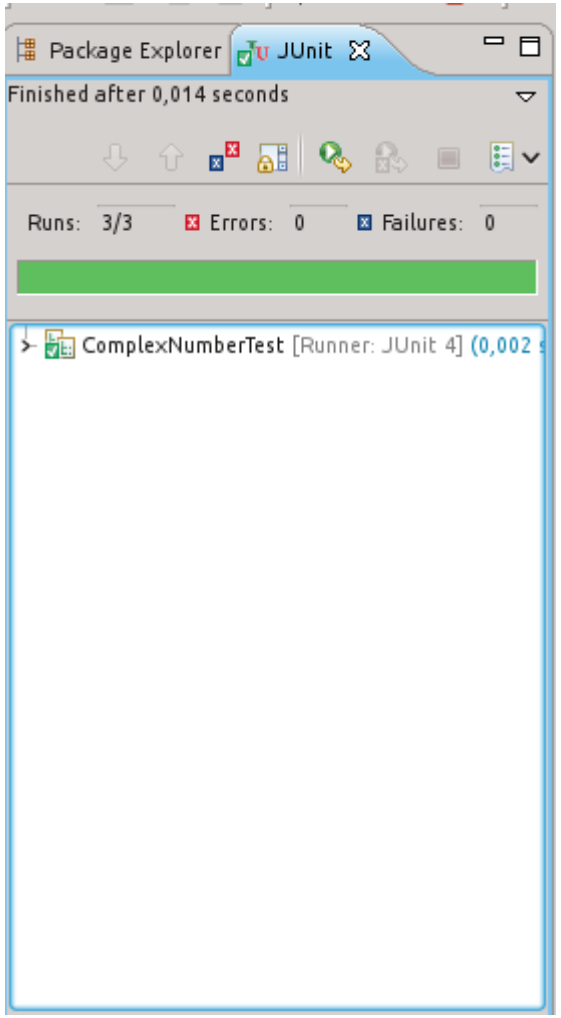
This method destroys the classes before each test is executed.

Now we can write unit tests:

```
@Test
public void testGetRealPart() {
  double expResult = 0.0;
  double result = cZeroOne.getRealPart();
  assertEquals(expResult, result);
}
@Test
public void testAdd() {
  ComplexNumber result = cZeroOne.add(cOneZero);
  assertEquals(cOneOne, result);
}
```

```
  @Test
  public void testMultiply() {
    ComplexNumber result = cZeroOne.multiply(cOneZero);
    assertEquals(cZeroOne, result);
  }
  @Test
  public void testEquals() {
  boolean expResult = false;
  boolean result = cZeroOne.equals(cOneZero);
  assertEquals(expResult, result);
}
```

We usually run the test with IDEs and we will see something similar to the following:



1 Introduction
2 Test Driven Development
3 Assertions
4 JUnit
5 First Example
6 Advanced Topics
7 Testing with Mock Objects
8 BDD and Cucumber
9 Web UI Testing
10 Code Coverage
11 Mutation Testing
12 Testing Strategy and Quality

# Suites

Run different sets of unit tests with the JUnit Platform suite engine.
Using `@Suite` annotation

```
import org.junit.platform.suite.api.IncludeTags;
import org.junit.platform.suite.api.SelectPackages;
import org.junit.platform.suite.api.Suite;

@SelectPackages({"uah.packageA"
                ,"uah.packageB"})

@IncludeTags("production")
@Suite
public class JUnitSuiteExample {
}
```

# Exceptions

The coefficient *a* must not be zero. Therefore, the following code:

`instance = new QuadraticEquation(0,1,1);`

will throw an exception when executing:

`boolean result = instance.checkCoefficientsAreOk();`

In JUnit Jupiter, this is tested with `assertThrows()`.

```
@Test
void testCheckCoefficientsAreOkException() {
  QuadraticEquation instance = new QuadraticEquation(0,1,1);

  assertThrows(CoefficientsException.class,
    () -> instance.checkCoefficientsAreOk());
}
```

# Testing Private Methods

In general private methods are not tested directly (they are tested indirectly through other public methods). However, if there is a need to do there are several alternatives:

- Change the visibility (make them public) temporarily. However, this changes the original program and does not follow the criteria of keeping tests and source code independently.
- Use Java reflection mechanisms

# Testing Private Methods - Example

```
public void testComputeDiscriminant() throws Exception {
    System.out.println("checkCoefficientsAreOk");
     Class c = null;
     Object ins = new QuadraticEquation(1,1,1);
     Method m = null;
     c = Class.forName("example.QuadraticEquation");
     m = c.getDeclaredMethod("computeDiscriminant");
     // Remove private access restriction:
     m.setAccessible(true);
     Object res = m.invoke(ins);
     Double result = (Double)res;
     System.out.println(result);
     double expectedResult = -3.0;
     assertEquals(result.doubleValue(), expectedResult);
}
```

# Parameterized tests

A parameterized test is an automated test in which the arguments change automatically.

# Parameterized tests - Example

```
import static org.junit.jupiter.api.Assertions.assertTrue;

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

public class QuadraticEquationParTest {

  @ParameterizedTest
  @ValueSource(doubles = {1.0, 2.0, 3.0, 4.0, 14.0})
  void checkCoefficientsAreOk(double coefficient) throws Exception {
    QuadraticEquation qe = new QuadraticEquation(coefficient, 6.0, 8.0);
    assertTrue(qe.checkCoefficientsAreOk());
  }
}
```

```
@ParameterizedTest(name = "a = {0} should be valid")
@ValueSource(doubles = {1.0, 2.0, 3.0, 4.0, 14.0})
void checkCoefficientsAreOk(double coefficient) throws Exception {
  QuadraticEquation qe = new QuadraticEquation(coefficient, 6.0, 8.0);
  assertTrue(qe.checkCoefficientsAreOk());
}
```

1. Introduction
2. Test Driven Development
3. Assertions
4. JUnit
5. First Example
6. Advanced Topics
7. Testing with Mock Objects
8. BDD and Cucumber
9. Web UI Testing
10. Code Coverage
11. Mutation Testing
12. Testing Strategy and Quality

# What are Mock objects?

Sometimes tests need to be carried out without having all objects implemented or the instantiation of all needed classes is very expensive (e.g., when dealing with databases)

Advantages of using mock objects include:

- Easy to instantiate
- If a CUT fails, the failure is located in the CUT, never in the mock!
- They allow us to reduce the testing time with simpler and faster classes

In Java, well-known frameworks include: Mockito, JMock and Easymock.

# Process

Process to create mock objects

1. Create the necessary Mock objects
2. Define their behaviour, establishing expectations
3. Create CUT instances using the mock objects
4. Run the test methods
5. Check with the mock objects if expectations were fulfilled.

"Mockito is a mocking framework that tastes really good. It lets you write beautiful tests with clean & simple API. Mockito doesn't give you hangover because the tests are very readable and they produce clean verification errors."

`https://site.mockito.org/`

With mock objects and Mockito we can do:

1. Test the behaviour
2. Force the behaviour
3. Use matchers in arguments passing
4. Call order verification
5. Etc.

# Installing Mockito

Mockito is usually used via Maven or Gradle to handle the dependencies.
`https://site.mockito.org/`
Manually, it is always possible to include the `.jar` file into your project.

1 Introduction
2 Test Driven Development
3 Assertions
4 JUnit
5 First Example
6 Advanced Topics
7 Testing with Mock Objects
8 BDD and Cucumber
9 Web UI Testing
10 Code Coverage
11 Mutation Testing
12 Testing Strategy and Quality

# What is BDD?

Behaviour-Driven Development (BDD) is an agile development practice that encourages collaboration between developers, QA, and business stakeholders.

- Derived from TDD — shifts focus from *how* to *what* the system should do.
- Tests are written in plain natural language close to acceptance criteria.
- Bridges the gap between business requirements and executable tests.
- The most popular BDD framework for Java is Cucumber.

**Key idea**

Scenarios written in *Gherkin* become both living documentation and automated tests.

# Gherkin Syntax

Gherkin is a plain-English DSL for describing expected behaviour.

```
Feature: Search UAH degrees

  Scenario: User navigates to degrees page
    Given the user is located in the index page
    When  the user enters studies section
    And   the user clicks in the degree section
    Then  the page with the degrees is shown
```

- `Feature` — groups related scenarios.
- `Scenario` — a single concrete test case.
- `Given` / `When` / `Then` / `And` — context, action, and expected outcome.

# Cucumber

`https://cucumber.io/`

- Reads `.feature` files written in Gherkin.
- Each step is matched by a step definition — a Java method annotated with `@Given`, `@When`, or `@Then`.
- Integrates with JUnit Platform via `cucumber-junit-platform-engine`.
- Can be combined with Selenium or Playwright to drive web UI scenarios.

# Cucumber – Maven Setup

```
<dependencies>
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-java</artifactId>
    <version>7.20.1</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-junit-platform-engine</artifactId>
    <version>7.20.1</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.junit.platform</groupId>
    <artifactId>junit-platform-suite</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Feature files must be placed under `src/test/resources/features/` so Maven copies them to the classpath.

# Cucumber – Step Definitions

```
public class SearchSteps {
  private WebDriver driver;

  @Given("the user is located in the index page")
  public void userIsInIndexPage() {
    driver = new FirefoxDriver();
    driver.navigate().to("https://uah.es/es/");
  }

  @When("the user enters studies section")
  public void userEnterStudySection() {
    driver.findElement(By.linkText("ESTUDIOS")).click();
  }

  @Then("the page with the degrees is shown")
  public void degreePage() {
    String title = driver.findElement(
                        By.linkText("Grados")).getText();
    Assertions.assertEquals("Grados", title);
  }
}
```

# Cucumber – JUnit Platform Runner

A plain class annotated with `@Suite` replaces the legacy `@RunWith(Cucumber.class)` runner.

```
@Suite
@IncludeEngines("cucumber")
@SelectClasspathResource("features")
@ConfigurationParameter(
    key   = "cucumber.glue",
    value = "steps")
public class RunCucumberTest {}
```

- `@SelectClasspathResource` points to the features folder on the classpath.
- `cucumber.glue` is the package containing step definitions.
- No `main` method or subclass needed.

1. Introduction
2. Test Driven Development
3. Assertions
4. JUnit
5. First Example
6. Advanced Topics
7. Testing with Mock Objects
8. BDD and Cucumber
9. Web UI Testing
10. Code Coverage
11. Mutation Testing
12. Testing Strategy and Quality

# Selenium WebDriver

https://www.selenium.dev/

- Industry-standard library for automating real browsers (Chrome, Firefox, Edge, Safari).
- Controls browsers through their native WebDriver protocols.
- Since **Selenium 4.6**, Selenium Manager is built in — no manual driver download needed.
- Widely used for end-to-end acceptance testing and regression testing.

# Selenium – Headless Example

```
@Test
void headlessChromeTest() {
  ChromeOptions options = new ChromeOptions();
  options.addArguments("--headless"); // no visible window

  WebDriver driver = new ChromeDriver(options);
  driver.get("https://escuelapolitecnica.uah.es/");
  System.out.println(driver.getTitle());
  driver.quit();
}
```

- Selenium Manager automatically downloads `chromedriver` on first run.
- Add `selenium-java:4.27.0` to the Maven dependencies.
- Run with: `mvn test -Drun.selenium=true`

# Why Playwright?

**Selenium WebDriver**

- Mature, large ecosystem
- Multiple language bindings
- Manual waits often needed
- Verbose setup

**Playwright**

- Auto-waiting built-in
- Browser binaries self-managed
- Modern, concise API
- Chromium, Firefox, WebKit

`https://playwright.dev/java/`

# Playwright Example (Java + JUnit)

```
@Test
void testUniversityLandingPageTitle() {
  Assumptions.assumeTrue(
    Boolean.parseBoolean(System.getProperty("run.playwright", "false")));

  try (Playwright pw = Playwright.create()) {
    Browser browser = pw.chromium().launch(
      new BrowserType.LaunchOptions().setHeadless(true));

    Page page = browser.newPage();
    page.navigate("https://escuelapolitecnica.uah.es/");
    assertTrue(page.title().toLowerCase().contains("escuela"));
    browser.close();
  }
}
```

Run with: `mvn test -Drun.playwright=true`

1. Introduction
2. Test Driven Development
3. Assertions
4. JUnit
5. First Example
6. Advanced Topics
7. Testing with Mock Objects
8. BDD and Cucumber
9. Web UI Testing
10. Code Coverage
11. Mutation Testing
12. Testing Strategy and Quality

# What is Code Coverage?

Code coverage is a metric that measures what percentage of the source code is exercised by the test suite.

- High coverage increases confidence that defects will be detected.
- Low coverage reveals areas of the code that are never tested.
- Coverage is **necessary but not sufficient** — 100% coverage does not guarantee correctness.

## Common coverage criteria

- Line / Statement coverage — was each executable statement reached?
- Branch coverage — was each branch of every conditional taken?
- Method coverage — was each method called at least once?

# JaCoCo

https://www.jacoco.org/

JaCoCo (Java Code Coverage) is the standard code-coverage library for Java.

- Instruments bytecode at runtime using a Java agent — no source changes needed.
- Generates reports in HTML, XML, and CSV.
- Integrates with Maven, Gradle, and CI platforms (SonarQube, GitHub Actions).
- Reports line, branch, method, class, and cyclomatic-complexity metrics.

# JaCoCo – Maven Configuration

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.14</version>
  <executions>
    <execution>
      <!-- attaches the JaCoCo agent before tests run -->
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals><goal>report</goal></goals>
    </execution>
  </executions>
</plugin>
```

# JaCoCo – Generating the Report

After adding the plugin, run:

```
mvn test
```

The HTML report is generated at:

```
target/site/jacoco/index.html
```

- Green lines — covered by tests.
- Red lines — not covered.
- Yellow diamonds — branch partially covered.
- The report shows per-class, per-method, and per-line metrics.

# JaCoCo – Enforcing a Minimum Coverage

JaCoCo can *fail the build* if coverage falls below a threshold:

```
<execution>
  <id>check</id>
  <goals><goal>check</goal></goals>
  <configuration>
    <rules>
      <rule>
        <element>BUNDLE</element>
        <limits>
          <limit>
            <counter>LINE</counter>
            <value>COVEREDRATIO</value>
            <minimum>0.80</minimum>
          </limit>
        </limits>
      </rule>
    </rules>
  </configuration>
</execution>
```

This enforces at least 80% line coverage across the project.

1. Introduction
2. Test Driven Development
3. Assertions
4. JUnit
5. First Example
6. Advanced Topics
7. Testing with Mock Objects
8. BDD and Cucumber
9. Web UI Testing
10. Code Coverage
11. Mutation Testing
12. Testing Strategy and Quality

# The Problem with Coverage

Code coverage measures *execution* — not *quality* of assertions.

**Counter-example: 100% line coverage, zero assertions**

```
@Test void testAdd() { calc.add(1, 1); }
```

Every line of `add()` is executed, but the result is never checked.

- A test that never asserts anything will achieve full coverage.
- Mutation testing solves this by verifying your tests actually *detect* faults.

# What is Mutation Testing?

## Idea

Automatically introduce a small mutation (bug) into the source code, then re-run the test suite. If at least one test fails, the mutant is killed. If all tests still pass, the mutant survived.

Common mutation operators:

- Replace + with −, ⋆, /.
- Replace > with >=, ==.
- Negate conditions: `if (b == 0)` → `if (b != 0)`.
- Remove a method call or `return` statement.

The mutation score $= \dfrac{\text{killed mutants}}{\text{total mutants}} \times 100\%$

# Mutation Score vs. Line Coverage

| Scenario | Line coverage | Mutation score |
|---|---|---|
| No assertions | 100% | ≈0% |
| Weak assertions | 90% | 40–60% |
| Strong assertions | 90% | >80% |

- High coverage + low mutation score ⇒ tests are *incomplete*.
- Aim for a mutation score $\geq 80\%$ for critical business logic.
- Some mutants are *equivalent* (semantically identical to the original) and cannot be killed — these are excluded from the score.

# PIT – Practical Mutation Testing for Java

`https://pitest.org/`

- PIT (Pitest) is the standard mutation testing tool for Java.
- Instruments bytecode — no source changes needed.
- Integrates with Maven and Gradle.
- Much faster than earlier tools: reruns only tests *relevant* to each mutant.
- Requires the `pitest-junit5-plugin` add-on for JUnit Jupiter/Platform.
- Generates an HTML report at `target/pit-reports/`.

# PIT – Maven Configuration

```
<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
  <version>1.17.3</version>
  <dependencies>
    <dependency>
      <groupId>org.pitest</groupId>
      <artifactId>pitest-junit5-plugin</artifactId>
      <version>1.2.1</version>
    </dependency>
  </dependencies>
  <configuration>
    <targetClasses>
      <param>calculator.*</param>
    </targetClasses>
    <targetTests>
      <param>calculator.*</param>
    </targetTests>
  </configuration>
</plugin>
```

**Note:** add `junit-platform-launcher` as a `test` dependency from the BOM to keep all JUnit Platform jars aligned; otherwise PIT's bundled launcher may conflict.

# PIT – Running and Reading the Report

Run mutation testing on the `PITExample`:

```
cd code/PITExample
mvn test-compile org.pitest:pitest-maven:mutationCoverage
```

Open the HTML report:

```
target/pit-reports/<timestamp>/index.html
```

- Green highlight — mutant was killed (test caught the bug).
- Red highlight — mutant survived (*test gap found*).
- Each survived mutant points to a missing or weak assertion.

# PIT – Example: Surviving Mutant

The `Calculator` class has a `subtract` method:

```
public double subtract(double a, double b) {
    return a - b;
}
```

`CalculatorTest` has no test for `subtract`. PIT will replace – with +:

```
// mutant: return a + b;
```

All tests still pass ⇒ mutant survived. PIT flags this as a test gap.
Running PIT on the `PITExample` produces: `Generated 11 mutations -- Killed 9 (82%)`
The 2 surviving mutants indicate missing assertions — adding a `subtract` test will push the score to 100%.

1. Introduction
2. Test Driven Development
3. Assertions
4. JUnit
5. First Example
6. Advanced Topics
7. Testing with Mock Objects
8. BDD and Cucumber
9. Web UI Testing
10. Code Coverage
11. Mutation Testing
12. Testing Strategy and Quality

# Testing Levels and Test Pyramid

Not all tests should run at the same level.

- Unit tests: fast, isolated, many.
- Integration tests: verify collaboration between components.
- End-to-end tests: validate user flows in a real environment.

**Test pyramid guideline**

Prefer many unit tests, fewer integration tests, and a small set of E2E tests.

- In this repository: `calculatorExample` and `complexNumber` are unit-focused.
- `CucumberTestExample`, `SeleniumTestExample`, and `PlaywrightExample` are UI/E2E-oriented.

# What Makes a Good Test? (FIRST + AAA)

**FIRST** properties:

- **F**ast
- **I**ndependent
- **R**epeatable
- **S**elf-validating
- **T**imely

Recommended structure: AAA (Arrange, Act, Assert)

```
@Test
void divideByZeroThrows() {
  // Arrange
  Calculator calc = new Calculator();
  // Act + Assert
  assertThrows(ArithmeticException.class,
      () -> calc.divide(1, 0));
}
```

# Designing Better Test Cases

Three techniques with strong payoff:

- Equivalence partitioning: divide input space into representative classes.
- Boundary value analysis: test min/max and just inside/outside.
- Positive and negative paths: valid scenarios and failure scenarios.

Example for `divide(a,b)`:

- Typical case: `divide(10, 2)`
- Boundary/precision case: `divide(10, 3)`
- Error path: `divide(1, 0)`

# Property-Based Testing

Instead of only hand-picked examples, define properties that should hold for many generated inputs.

```
@Property
void addIsCommutative(@ForAll int a, @ForAll int b) {
    assertEquals(calc.add(a, b), calc.add(b, a));
}

@Property
void addZeroIsIdentity(@ForAll int a) {
    assertEquals(a, calc.add(a, 0));
}
```

- On failure, the framework applies shrinking to find the smallest counterexample.
- Java: **jqwik**. Python: **Hypothesis**.
- Useful for algebraic laws, invariants, and round-trip properties.

# Avoiding Flaky Tests

Flaky tests fail intermittently without code changes.

- Avoid fixed sleeps; use explicit waits or Playwright auto-waiting.
- Keep test data deterministic and isolated.
- Do not depend on execution order.
- Control environment differences (browser, locale, timezone).
- For optional local browser runs, gate tests with: `-Drun.selenium=true`, `-Drun.playwright=true`.

# Test Doubles: Vocabulary

- Dummy: passed around, never used.
- Stub: provides predefined responses.
- Mock: verifies interactions (calls, arguments, order).
- Spy: wraps a real object and observes calls.
- Fake: simplified working implementation (in-memory DB, etc.).

Use doubles to isolate the class under test and speed up feedback.
In this repository:

- `MockitoATM` demonstrates Mockito-based mocks.
- `EasyMockExample` demonstrates EasyMock interactions.

# CI Testing Strategy

Suggested cadence in CI:

1. Every commit: unit tests + JaCoCo.
2. Pull requests: unit + selected UI smoke tests.
3. Nightly or pre-release: full UI + PIT mutation testing.

**Why this split?**

Fast feedback for developers, deeper quality checks on a scheduled pipeline.

# CI Commands

Use lightweight commands per stage:

```
# fast feedback (every commit)
cd code/calculatorExample
mvn test

# deeper feedback (scheduled)
cd code/PITExample
mvn test-compile org.pitest:pitest-maven:mutationCoverage
```

# Automated Testing: What to Automate First

Automation should start with checks that are frequent, fast, and objective:

- Run unit tests on every commit or before every push.
- Automate repeatable quality gates: compilation, linting, and coverage thresholds.
- Keep the fast test suite deterministic to avoid noisy failures.
- Reserve slower end-to-end or mutation suites for scheduled runs.

**Rule of thumb**

Automate execution aggressively; automate judgment carefully.

# AI/Agents in Testing: Useful and Safe

AI tools can accelerate testing work, but they do not replace engineering judgment.

- Good use cases: generate test skeletons, propose edge cases, suggest property tests.
- Human review is mandatory: validate assertions, names, and intent.
- Watch for risks: plausible but wrong tests, over-mocking, brittle test logic.

Practical policy:

- Automate execution: yes.
- Automate authoring: partially.
- Automate final quality decisions: no.

# Summary

JUnit (or an equivalent framework) is standard practice in Java testing.
It can be used in combination with other frameworks such as Mockito, EasyMock, etc.
It should be part of the software development life cycle with:

- Continuous integration
- Configuration management, e.g., Git.
- etc.